

Buchtausch-App — Phase 2: Erstellungsphase

1. Tables

1.1. Table Genres

1.1.1. Create Table

```
CREATE TABLE IF NOT EXISTS genres (  
  genre_id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL UNIQUE  
);
```

1.1.2. Dummy Daten

```
INSERT INTO genres  
(  
  name  
)  
VALUES ('Novel'),  
( 'Science Fiction'),  
( 'Crime'),  
( 'Fantasy'),  
( 'Non-Fiction'),  
( 'Biography'),  
( 'Thriller'),  
( 'Historical Fiction'),  
( 'Children''s Book'),  
( 'Poetry'),  
( 'Humor'),  
( 'Travel Guide');
```

1.2. Table languages

1.2.1. Create Table

```
CREATE TABLE IF NOT EXISTS languages (  
  language_id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL UNIQUE,  
  code CHAR(2) NOT NULL UNIQUE  
);
```

1.2.2. Dummy Daten

```
INSERT INTO languages  
(  
  name,  
  code  
)  
VALUES  
( 'German', 'de'),  
( 'English', 'en'),  
( 'French', 'fr'),  
( 'Spanish', 'es'),  
( 'Italian', 'it'),  
( 'Dutch', 'nl'),  
( 'Polish', 'pl'),  
( 'Russian', 'ru'),  
( 'Turkish', 'tr'),  
( 'Portuguese', 'pt'),  
( 'Swedish', 'sv'),  
( 'Japanese', 'ja');
```

1.3. Table publishers

1.3.1. Create Table

```
CREATE TABLE IF NOT EXISTS publishers (  
    publisher_id SERIAL PRIMARY KEY,  
    name VARCHAR(200) NOT NULL,  
    country VARCHAR(100)  
);
```

1.3.2. Dummy Daten

```
INSERT INTO publishers  
(  
    name,  
    country  
)  
VALUES  
(  
    'Suhrkamp Verlag', 'Germany'),  
('C.H. Beck', 'Germany'),  
('Rowohlt Verlag', 'Germany'),  
('Penguin Books', 'Great Britain'),  
('Random House', 'USA'),  
('Hanser Verlag', 'Germany'),  
('dtv Verlagsgesellschaft', 'Germany'),  
('S. Fischer Verlag', 'Germany'),  
('Piper Verlag', 'Germany'),  
('Ullstein Buchverlage', 'Germany'),  
('Kiepenheuer & Witsch', 'Germany'),  
('Bloomsbury Publishing', 'Great Britain');
```

1.4. Table book_conditions

1.4.1. Create Table

```
CREATE TABLE IF NOT EXISTS book_conditions (  
    condition_id SERIAL PRIMARY KEY,  
    label VARCHAR(50) NOT NULL,  
    description TEXT  
);
```

1.4.2. Dummy Daten

```
INSERT INTO book_conditions  
(  
    label,  
    description  
)  
VALUES  
(  
    'New', 'Unread, perfect condition'),  
('Very Good', 'Barely any wear, like new'),  
('Good', 'Light signs of use, fully readable'),  
('Acceptable', 'Visible signs of use, but complete'),  
('Sufficient', 'Heavy signs of use, possible annotations'),  
('Poor', 'Significant damage, but readable'),  
('Defective', 'Pages missing or severely damaged');
```

1.5. Table users

1.5.1. Create Table

```
CREATE TABLE IF NOT EXISTS users (  
    user_id SERIAL PRIMARY KEY,  
    first_name VARCHAR(100) NOT NULL,  
    last_name VARCHAR(100) NOT NULL,  
    email VARCHAR(200) NOT NULL UNIQUE,  
    phone VARCHAR(30),  
    street VARCHAR(200),  
    city VARCHAR(100),
```

```

postal_code    VARCHAR(20),
country        VARCHAR(100) DEFAULT 'Germany',
latitude       DECIMAL(9,6),
longitude      DECIMAL(9,6),
registered_at  TIMESTAMP NOT NULL DEFAULT NOW(),
is_active      BOOLEAN NOT NULL DEFAULT TRUE
);

```

1.5.2 Dummy Daten

```

INSERT INTO users (first_name, last_name, email, phone, street, city, postal_code,
country, latitude, longitude)
VALUES
(
  'Anna', 'Müller', 'anna.mueller@example.de', '+49 151 11111111', 'Hauptstraße 1',
  'Berlin', '10115', 'Germany', 52.520008, 13.404954
),
(
  'Ben', 'Schmidt', 'ben.schmidt@example.de', '+49 152 22222222', 'Goethestraße 12',
  'München', '80336', 'Germany', 48.135125, 11.581981
),
(
  'Clara', 'Fischer', 'clara.fischer@example.de', '+49 153 33333333', 'Schillerplatz 3',
  'Hamburg', '20095', 'Germany', 53.550341, 9.992862
),
(
  'David', 'Weber', 'david.weber@example.de', '+49 154 44444444', 'Bahnhofstraße 7',
  'Frankfurt', '60329', 'Germany', 50.110922, 8.682127
),
(
  'Eva', 'Wagner', 'eva.wagner@example.de', '+49 155 55555555', 'Mozartstraße 9',
  'Stuttgart', '70173', 'Germany', 48.775846, 9.182932
),
(
  'Felix', 'Becker', 'felix.becker@example.de', '+49 156 66666666', 'Rüdesheimer Str. 5',
  'Köln', '50667', 'Germany', 50.938361, 6.959974
),
(
  'Greta', 'Hoffmann', 'greta.hoffmann@example.de', '+49 157 77777777', 'Am Markt 2',
  'Düsseldorf', '40213', 'Germany', 51.225402, 6.776314
),
(
  'Hans', 'Schäfer', 'hans.schaefer@example.de', '+49 158 88888888', 'Lindenallee 14',
  'Leipzig', '04109', 'Germany', 51.339695, 12.373075
),
(
  'Ida', 'Koch', 'ida.koch@example.de', '+49 159 99999999', 'Friedrichstraße 22',
  'Dresden', '01067', 'Germany', 51.050409, 13.737262
),
(
  'Jonas', 'Richter', 'jonas.richter@example.de', '+49 160 10101010', 'Parkstraße 8',
  'Bremen', '28195', 'Germany', 53.079296, 8.801694
),
(
  'Katja', 'Klein', 'katja.klein@example.de', '+49 161 11111112', 'Bergstraße 19',
  'Hannover', '30159', 'Germany', 52.374478, 9.738553
),
(
  'Lars', 'Wolf', 'lars.wolf@example.de', '+49 162 22222223', 'Seestraße 6', 'Nürnberg',
  '90402', 'Germany', 49.452030, 11.076750
);

```

1.6. Table books

1.6.1. Create Table

```
CREATE TABLE IF NOT EXISTS books (  
  book_id          SERIAL PRIMARY KEY,  
  isbn             VARCHAR(20) UNIQUE,  
  title            VARCHAR(300) NOT NULL,  
  -- author bleibt als Fallback für Legacy-Daten, neu: author_id FK  
  author           VARCHAR(200),  
  author_id        INT REFERENCES authors(author_id),  
  publisher_id     INT REFERENCES publishers(publisher_id),  
  genre_id         INT REFERENCES genres(genre_id),  
  language_id      INT REFERENCES languages(language_id),  
  year_published   SMALLINT,  
  description       TEXT  
);
```

1.6.2. Dummy Daten

```
INSERT INTO books (isbn, title, author, author_id, publisher_id, genre_id, language_id,  
year_published, description)  
VALUES  
(  
  '978-3-518-41404-0', 'Steppenwolf', 'Hermann Hesse', 1, 1, 1, 1, 1927, 'A German  
literary classic exploring alienation, identity, and the duality of human nature.'  
),  
(  
  '978-3-423-13522-3', 'Perfume: The Story of a Murderer', 'Patrick Süskind', 2, 7, 1, 1,  
1985, 'A darkly obsessive tale of a murderer in 18th-century France driven by an  
extraordinary sense of smell.'  
),  
(  
  '978-3-499-22765-5', 'The Name of the Rose', 'Umberto Eco', 3, 3, 3, 1, 1986, 'A  
medieval monastery mystery steeped in symbolism, philosophy, and theological intrigue.'  
),  
(  
  '978-0-7432-7356-5', 'The Hitchhiker's Guide to the Galaxy', 'Douglas Adams', 4, 5, 2,  
2, 1979, 'A wildly comic science fiction adventure following the accidental destruction  
of Earth.'  
),  
(  
  '978-3-257-06401-1', 'Bahnwärter Thiel', 'Gerhart Hauptmann', 5, 6, 8, 1, 1888, 'A  
German novella exploring guilt, fate, and the overwhelming force of nature.'  
),  
(  
  '978-3-446-20387-6', 'Homo Faber', 'Max Frisch', 6, 6, 1, 1, 1957, 'A rationalist  
engineer confronts fate and emotion in this landmark of modern German-language  
literature.'  
),  
(  
  '978-3-10-397077-8', 'Sophie's World', 'Jostein Gaarder', 7, 8, 5, 1, 1993, 'A history  
of Western philosophy told as a gripping coming-of-age novel.'  
),  
(  
  '978-3-453-31628-0', 'Dune', 'Frank Herbert', 8, 7, 2, 1, 2019, 'An epic science  
fiction saga of power, religion, and ecology set on a desert planet.'  
),  
(  
  '978-3-596-29752-4', 'The Metamorphosis', 'Franz Kafka', 9, 7, 1, 1, 1915, 'Kafka at  
his most iconic – a man wakes up transformed into an insect in this unsettling  
masterpiece.'  
),  
(  
  '978-3-423-21190-4', 'Buddenbrooks', 'Thomas Mann', 10, 7, 8, 1, 1901, 'A sweeping  
family saga tracing the decline of a wealthy merchant dynasty over four generations.'  
),
```

```
(
  '978-3-8321-6065-0', 'The Alchemist', 'Paulo Coelho', 11, 7, 1, 1, 1993, 'A
philosophical parable about following your dreams and discovering your personal
destiny.'
),
(
  '978-0-06-112008-4', 'To Kill a Mockingbird', 'Harper Lee', 12, 5, 1, 2, 1960, 'A
landmark American novel confronting racial injustice and moral courage in the Deep
South.'
);
```

1.7. Table book_copies

1.7.1. Create Table

```
CREATE TABLE IF NOT EXISTS book_copies (
  copy_id          SERIAL PRIMARY KEY,
  book_id          INT NOT NULL REFERENCES books(book_id),
  owner_id         INT NOT NULL REFERENCES users(user_id),
  condition_id     INT NOT NULL REFERENCES book_conditions(condition_id),
  max_loan_days    SMALLINT NOT NULL DEFAULT 14,
  allow_post       BOOLEAN NOT NULL DEFAULT FALSE,
  pickup_address   VARCHAR(300),
  pickup_latitude  DECIMAL(9,6),
  pickup_longitude DECIMAL(9,6),
  is_available     BOOLEAN NOT NULL DEFAULT TRUE,
  added_at        TIMESTAMP NOT NULL DEFAULT NOW()
);
```

1.7.2. Dummy Daten

```
INSERT INTO book_copies
(
  book_id,
  owner_id,
  condition_id,
  max_loan_days,
  allow_post,
  pickup_address,
  pickup_latitude,
  pickup_longitude,
  is_available
)
VALUES
(1, 1, 2, 14, FALSE, 'Hauptstraße 1, 10115 Berlin', 52.520008, 13.404954, TRUE),
(2, 1, 2, 14, FALSE, 'Hauptstraße 1, 10115 Berlin', 52.520008, 13.404954, TRUE),
(2, 2, 3, 21, TRUE, 'Goethestraße 12, 80336 München', 48.135125, 11.581981, TRUE),
(3, 3, 1, 14, FALSE, 'Schillerplatz 3, 20095 Hamburg', 53.550341, 9.992862, TRUE),
(4, 4, 4, 28, TRUE, 'Bahnhofstraße 7, 60329 Frankfurt', 50.110922, 8.682127, FALSE),
(5, 5, 2, 14, FALSE, 'Mozartstraße 9, 70173 Stuttgart', 48.775846, 9.182932, TRUE),
(6, 6, 3, 14, TRUE, 'Rüdesheimer Str. 5, 50667 Köln', 50.938361, 6.959974, TRUE),
(7, 7, 1, 21, FALSE, 'Am Markt 2, 40213 Düsseldorf', 51.225402, 6.776314, TRUE),
(8, 8, 2, 14, TRUE, 'Lindenallee 14, 04109 Leipzig', 51.339695, 12.373075, FALSE),
(9, 9, 5, 14, FALSE, 'Friedrichstraße 22, 01067 Dresden', 51.050409, 13.737262, TRUE),
(10, 10, 3, 28, TRUE, 'Parkstraße 8, 28195 Bremen', 53.079296, 8.801694, TRUE),
(11, 11, 2, 14, FALSE, 'Bergstraße 19, 30159 Hannover', 52.374478, 9.738553, TRUE),
(12, 12, 1, 21, TRUE, 'Seestraße 6, 90402 Nürnberg', 49.452030, 11.076750, TRUE);
```

1.8. Table time_slots

1.8.1. Create Table

```
CREATE TABLE IF NOT EXISTS time_slots (  
    slot_id          SERIAL PRIMARY KEY,  
    copy_id          INT NOT NULL REFERENCES book_copies(copy_id),  
    day_of_week       SMALLINT NOT NULL CHECK (day_of_week BETWEEN 0 AND 6),  
    start_time        TIME NOT NULL,  
    end_time          TIME NOT NULL,  
    CONSTRAINT chk_slot_times CHECK (end_time > start_time)  
);
```

1.8.2. Dummy Daten

```
INSERT INTO time_slots  
(  
    copy_id,  
    day_of_week,  
    start_time,  
    end_time  
)  
VALUES  
(1, 1, '09:00', '12:00'),  
(1, 3, '14:00', '17:00'), -- Exemplar 1 hat jetzt 2 Zeitslots  
(2, 2, '14:00', '17:00'),  
(3, 3, '10:00', '13:00'),  
(4, 4, '15:00', '18:00'),  
(5, 5, '09:00', '11:00'),  
(6, 6, '11:00', '14:00'),  
(7, 0, '10:00', '15:00'),  
(8, 1, '16:00', '19:00'),  
(9, 2, '08:00', '10:00'),  
(10, 3, '13:00', '16:00'),  
(11, 4, '17:00', '20:00'),  
(12, 5, '09:00', '12:00'),  
(12, 0, '10:00', '13:00'); -- Exemplar 12 hat jetzt 2 Zeitslots
```

1.9. Table loans

1.9.1. Create Table

```
CREATE TABLE IF NOT EXISTS loans (  
    loan_id          SERIAL PRIMARY KEY,  
    copy_id          INT NOT NULL REFERENCES book_copies(copy_id),  
    borrower_id       INT NOT NULL REFERENCES users(user_id),  
    slot_id           INT REFERENCES time_slots(slot_id),  
    loaned_at         DATE NOT NULL DEFAULT CURRENT_DATE,  
    due_date          DATE NOT NULL,  
    returned_at       DATE,  
    is_via_post        BOOLEAN NOT NULL DEFAULT FALSE,  
    status            VARCHAR(20) NOT NULL DEFAULT 'active'  
    CHECK (status IN ('active', 'returned', 'overdue', 'cancelled'))  
);
```

1.9.2. Dummy Daten

```
INSERT INTO loans  
(  
    copy_id,  
    borrower_id,  
    slot_id,  
    loaned_at,  
    due_date,  
    returned_at,  
    is_via_post,  
    status  
)
```

```
VALUES
(1, 2, 1, '2024-01-10', '2024-01-24', '2024-01-22', FALSE, 'returned'),
(2, 3, 2, '2024-02-05', '2024-02-26', '2024-02-24', TRUE, 'returned'),
(3, 4, 3, '2024-03-01', '2024-03-15', '2024-03-14', FALSE, 'returned'),
(5, 6, 5, '2024-04-10', '2024-04-24', '2024-04-23', FALSE, 'returned'),
(6, 7, 6, '2024-05-15', '2024-05-29', '2024-05-30', FALSE, 'returned'),
(7, 8, 7, '2024-06-01', '2024-06-22', NULL, FALSE, 'overdue'),
(9, 10, 9, '2024-07-20', '2024-08-03', '2024-08-01', FALSE, 'returned'),
(10, 1, 10, '2024-08-05', '2024-09-02', NULL, TRUE, 'active'),
(11, 2, 11, '2024-08-20', '2024-09-03', NULL, FALSE, 'active'),
(12, 3, 12, '2024-09-01', '2024-09-22', NULL, TRUE, 'active'),
(1, 4, 1, '2024-09-10', '2024-09-24', NULL, FALSE, 'cancelled'),
(3, 5, 3, '2024-10-01', '2024-10-15', NULL, FALSE, 'active');
```

1.10. Table ratings

1.10.1. Create Table

```
CREATE TABLE IF NOT EXISTS ratings (
    rating_id SERIAL PRIMARY KEY,
    loan_id INT NOT NULL UNIQUE REFERENCES loans(loan_id),
    reviewer_id INT NOT NULL REFERENCES users(user_id),
    book_id INT NOT NULL REFERENCES books(book_id),
    score SMALLINT NOT NULL CHECK (score BETWEEN 1 AND 5),
    comment TEXT,
    rated_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

1.10.2. Dummy Daten

```
INSERT INTO ratings
(
    loan_id,
    reviewer_id,
    book_id,
    score,
    comment
)
VALUES
(1, 2, 1, 5, 'Absolutes Meisterwerk, sehr gerne weiterempfohlen!'),
(2, 3, 2, 4, 'Fesselnd und gut geschrieben, toller Zustand des Buches.'),
(3, 4, 3, 5, 'Spannend von Anfang bis Ende – ein Genuss!'),
(4, 6, 5, 3, 'Interessant, aber etwas langatmig an manchen Stellen.'),
(5, 7, 6, 4, 'Sehr philosophisch, regt zum Nachdenken an.'),
(7, 10, 9, 4, 'Kafka in Bestform – kurz aber intensiv.'),
(9, 1, 11, 5, 'Wunderschöne Geschichte, absolut lesenswert.'),
(10, 2, 10, 4, 'Großartiger Klassiker, gut erhalten.'),
(11, 3, 12, 5, 'Bewegender Roman, gehört in jedes Bücherregal.'),
(12, 5, 3, 4, 'Tolle Lektüre, Buch war in sehr gutem Zustand.');
```

1.11. Table messages

1.11.1. Create Table

```
CREATE TABLE IF NOT EXISTS messages (
    message_id SERIAL PRIMARY KEY,
    sender_id INT NOT NULL REFERENCES users(user_id),
    receiver_id INT NOT NULL REFERENCES users(user_id),
    copy_id INT REFERENCES book_copies(copy_id),
    subject VARCHAR(200),
    body TEXT NOT NULL,
    sent_at TIMESTAMP NOT NULL DEFAULT NOW(),
    is_read BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT chk_different_users CHECK (sender_id <> receiver_id)
);
```

1.11.2. Dummy Daten

```
INSERT INTO messages
(
    sender_id,
    receiver_id,
    copy_id,
    subject,
    body,
    is_read
)
VALUES
(2, 1, 1, 'Anfrage: Der Steppenwolf', 'Hallo Anna, ist das Buch noch verfügbar? Ich würde es gerne ausleihen.', TRUE),
(1, 2, 1, 'Re: Anfrage: Der Steppenwolf', 'Hallo Ben, ja klar! Du kannst es nächsten Montag abholen.', TRUE),
(3, 2, 2, 'Buchzustand Das Parfum', 'Hi Ben, wie ist der aktuelle Zustand des Buches?', FALSE),
(4, 3, 3, 'Abholung Der Name der Rose', 'Wann kann ich das Buch abholen?', TRUE),
(5, 4, 4, 'Versand möglich?', 'Hi David, kannst du das Buch auch per Post schicken?', FALSE),
(6, 5, 5, 'Verlängerung möglich?', 'Hallo Eva, kann ich die Leihfrist um eine Woche verlängern?', TRUE),
(7, 6, 6, 'Danke für das Buch', 'Hallo Felix, das Buch war super. Vielen Dank!', TRUE),
(8, 7, 7, 'Sofies Welt – Verfügbarkeit', 'Ist das Buch nächste Woche verfügbar?', FALSE),
(9, 8, 8, 'Dune – Rückgabe', 'Hi Hans, ich gebe das Buch übermorgen zurück. Passt das?', TRUE),
(10, 9, 9, 'Buchempfehlung', 'Hallo Ida, die Verwandlung war wirklich beeindruckend!', FALSE),
(11, 10, 10, 'Leihfrist Buddenbrooks', 'Guten Tag Jonas, reichen mir 4 Wochen Leihzeit?', TRUE),
(12, 1, 11, 'Abholung in Hannover', 'Hallo Katja, wann kann ich den Alchimisten abholen?', FALSE);
```

1.12. Table wishlist

1.12.1. Create Table

```
CREATE TABLE IF NOT EXISTS wishlist (
    wishlist_id          SERIAL PRIMARY KEY,
    user_id              INT NOT NULL REFERENCES users(user_id),
    book_id              INT NOT NULL REFERENCES books(book_id),
    added_at             TIMESTAMP NOT NULL DEFAULT NOW(),
    UNIQUE(user_id, book_id)
);
```

1.12.2. Dummy Daten

```
INSERT INTO wishlist
(
    user_id,
    book_id
)
VALUES
(1, 4),
(1, 8),
(2, 3),
(2, 9),
(3, 1),
(4, 2),
(5, 6),
(6, 7),
(7, 10),
(8, 11),
(9, 12),
(10, 5),
(11, 4),
(12, 1);
```


1.13. Table authors

1.13.1. Create Table

```
CREATE TABLE IF NOT EXISTS authors (  
  author_id SERIAL PRIMARY KEY,  
  first_name VARCHAR(100) NOT NULL,  
  last_name VARCHAR(100) NOT NULL,  
  birth_year SMALLINT,  
  nationality VARCHAR(100)  
);
```

1.12.2. Dummy Daten

```
INSERT INTO authors  
(  
  first_name,  
  last_name, birth_year, nationality  
)  
VALUES  
(  
  'Hermann', 'Hesse', 1877, 'German'),  
  ('Patrick', 'Süskind', 1949, 'German'),  
  ('Umberto', 'Eco', 1932, 'Italian'),  
  ('Douglas', 'Adams', 1952, 'British'),  
  ('Gerhart', 'Hauptmann', 1862, 'German'),  
  ('Max', 'Frisch', 1911, 'Swiss'),  
  ('Jostein', 'Gaarder', 1952, 'Norwegian'),  
  ('Frank', 'Herbert', 1920, 'American'),  
  ('Franz', 'Kafka', 1883, 'Bohemian'),  
  ('Thomas', 'Mann', 1875, 'German'),  
  ('Paulo', 'Coelho', 1947, 'Brazilian'),  
  ('Harper', 'Lee', 1926, 'American');
```

1.13. Table book_authors

1.13.1. Create Table

```
CREATE TABLE IF NOT EXISTS book_authors (  
  book_id INT NOT NULL REFERENCES books(book_id) ON DELETE CASCADE,  
  author_id INT NOT NULL REFERENCES authors(author_id) ON DELETE CASCADE,  
  PRIMARY KEY (book_id, author_id)  
);
```

1.12.2. Dummy Daten

```
INSERT INTO book_authors (book_id, author_id)  
SELECT b.book_id, b.author_id FROM books b WHERE b.author_id IS NOT NULL;
```

2. Testfälle

2.1. Testfall 1

```
-- -----
-- Testfall 1.1: users -> book_copies (1:N)
-- Ein Nutzer kann mehrere Exemplare besitzen.
-- Erwartung: mindestens ein Nutzer mit mehr als 1 Exemplar.
-- -----
DO $$ BEGIN RAISE NOTICE '--- Testfall 1.1: users -> book_copies (1:N) ---'; END $$;

SELECT
    u.user_id,
    u.first_name || ' ' || u.last_name AS nutzer,
    COUNT(bc.copy_id)                  AS anzahl_exemplare
FROM users u
JOIN book_copies bc ON bc.owner_id = u.user_id
GROUP BY u.user_id, u.first_name, u.last_name
ORDER BY anzahl_exemplare DESC
LIMIT 5;

DO $$
DECLARE
    v_multiple INT;
    v_result   INT;
BEGIN
    SELECT COUNT(*) INTO v_multiple
    FROM (
        SELECT u.user_id, COUNT(bc.copy_id) AS anzahl_exemplare
        FROM users u
        JOIN book_copies bc ON bc.owner_id = u.user_id
        GROUP BY u.user_id
        HAVING COUNT(bc.copy_id) > 1
    ) sub;

    SELECT COUNT(DISTINCT owner_id) INTO v_result FROM book_copies;

    RAISE NOTICE 'Testfall 1.1: Nutzer mit Exemplaren: %, davon mit >1 Exemplar: %',
v_result, v_multiple;

    IF v_multiple > 0 THEN
        RAISE NOTICE 'Testfall 1.1: PASS -- % Nutzer besitzen mehr als 1 Exemplar (1:N
bestätigt)', v_multiple;
    ELSE
        RAISE NOTICE 'Testfall 1.1: HINWEIS -- Jeder Nutzer hat genau 1 Exemplar in
Testdaten (1:N möglich, aber nicht belegt)';
    END IF;
END $$;
```

Output

123 user_id	A-Z user	123 number_of_copies
1	Anna Müller	2
9	Ida Koch	1
7	Greta Hoffmann	1
6	Felix Becker	1
10	Jonas Richter	1

2.2. Testfall 2

```
-----
-- Testfall 1.2: books -> book_copies (1:N)
-----
DO $$ BEGIN RAISE NOTICE '--- Testfall 1.2: books -> book_copies (1:N) ---'; END $$;

SELECT
    b.book_id,
    b.title,
    COUNT(bc.copy_id) AS anzahl_exemplare
FROM books b
JOIN book_copies bc ON bc.book_id = b.book_id
GROUP BY b.book_id, b.title
ORDER BY anzahl_exemplare DESC
LIMIT 5;

DO $$
DECLARE
    v_multiple INT;
    v_result   INT;
BEGIN
    SELECT COUNT(*) INTO v_multiple
    FROM (
        SELECT b.book_id, COUNT(bc.copy_id) AS anzahl_exemplare
        FROM books b
        JOIN book_copies bc ON bc.book_id = b.book_id
        GROUP BY b.book_id
        HAVING COUNT(bc.copy_id) > 1
    ) sub;

    -- Gesamtanzahl Titel mit mind. 1 Exemplar
    SELECT COUNT(DISTINCT book_id) INTO v_result FROM book_copies;

    RAISE NOTICE 'Testfall 1.2: Gesamt Titel mit Exemplaren: %, davon mit >1 Exemplar:
%', v_result, v_multiple;

    IF v_multiple > 0 THEN
        RAISE NOTICE 'Testfall 1.2: PASS -- % Titel haben anzahl_exemplare > 1 (1:N
bestätigt)', v_multiple;
    ELSE
        RAISE NOTICE 'Testfall 1.2: HINWEIS -- Jeder Titel hat genau 1 Exemplar in
Testdaten (1:N möglich, aber nicht belegt)';
    END IF;
END $$;
```

Ouput

123 book_id	A-Z title	123 number_of_copies
2	Perfume: The Story of a Murderer	2
6	Homo Faber	1
10	Buddenbrooks	1
11	The Alchemist	1
4	The Hitchhiker's Guide to the Gala	1

2.3. Testfall 3

```
-- -----  
-- Testfall 1.3: book_copies -> time_slots (1:N)  
-- -----  
DO $$ BEGIN RAISE NOTICE '--- Testfall 1.3: book_copies -> time_slots (1:N) ---'; END  
$$;  
  
SELECT  
    bc.copy_id,  
    b.title,  
    COUNT(ts.slot_id) AS number_of_slots  
FROM book_copies bc  
JOIN books b ON b.book_id = bc.book_id  
JOIN time_slots ts ON ts.copy_id = bc.copy_id  
GROUP BY bc.copy_id, b.title  
ORDER BY number_of_slots DESC  
LIMIT 5;  
  
DO $$  
DECLARE  
    v_multiple INT;  
    v_result INT;  
BEGIN  
    SELECT COUNT(*) INTO v_multiple  
    FROM (  
        SELECT bc.copy_id, COUNT(ts.slot_id) AS number_of_slots  
        FROM book_copies bc  
        JOIN time_slots ts ON ts.copy_id = bc.copy_id  
        GROUP BY bc.copy_id  
        HAVING COUNT(ts.slot_id) > 1  
    ) sub;  
  
    SELECT COUNT(DISTINCT copy_id) INTO v_result FROM time_slots;  
  
    RAISE NOTICE 'Testfall 1.3: Exemplare mit Zeitslots: %, davon mit >1 Slot: %',  
v_result, v_multiple;  
  
    IF v_multiple > 0 THEN  
        RAISE NOTICE 'Testfall 1.3: PASS -- % Exemplare haben mehrere Zeitslots (1:N  
bestätigt)', v_multiple;  
    ELSE  
        RAISE NOTICE 'Testfall 1.3: HINWEIS -- Jedes Exemplar hat genau 1 Zeitslot in  
Testdaten (1:N möglich, aber nicht belegt)';  
    END IF;  
END $$;
```

Output

123 copy_id ▼	A-Z title ▼	123 number_of_slots ▼
9	The Metamorphosis	1
5	Bahnwärter Thiel	1
8	Dune	1
3	The Name of the Rose	1
1	Steppenwolf	1

2.4. Testfall 4

```
-----  
-- Testfall 1.4: loans -> ratings (1:1, UNIQUE)  
-----  
DO $$ BEGIN RAISE NOTICE '--- Testfall 1.4: loans -> ratings (1:1, UNIQUE) ---'; END $$;  
  
SELECT  
    l.loan_id,  
    COUNT(r.rating_id) AS number_of_ratings  
FROM loans l  
LEFT JOIN ratings r ON r.loan_id = l.loan_id  
GROUP BY l.loan_id  
ORDER BY number_of_ratings DESC  
LIMIT 5;  
  
DO $$  
DECLARE v INT;  
BEGIN  
    SELECT COUNT(*) INTO v  
    FROM (  
        SELECT loan_id FROM ratings GROUP BY loan_id HAVING COUNT(*) > 1  
    ) sub;  
    IF v = 0 THEN  
        RAISE NOTICE 'Testfall 1.4: PASS -- Keine loan_id hat mehr als 1 Bewertung (1:1  
bestätigt)';  
    ELSE  
        RAISE NOTICE 'Testfall 1.4: FAIL -- % loan_ids haben mehr als 1 Bewertung!', v;  
    END IF;  
END $$;
```

Output

123 loan_id	123 number_of_ratings
10	1
4	1
2	1
11	1
9	1

2.5. Testfall 5

```
-----  
-- Testfall 2.1: Wunschliste -- jede Kombination nur einmal  
-----  
DO $$ BEGIN RAISE NOTICE '--- Testfall 2.1: wishlist UNIQUE(user_id, book_id) ---'; END  
$$;  
  
SELECT  
    u.first_name || ' ' || u.last_name AS user,  
    COUNT(w.book_id) AS wishlist_entry  
FROM users u  
JOIN wishlist w ON w.user_id = u.user_id  
GROUP BY u.user_id, u.first_name, u.last_name  
ORDER BY wishlist_entry DESC;  
  
DO $$  
DECLARE v INT;  
BEGIN  
    SELECT COUNT(*) INTO v  
    FROM (  
        SELECT user_id, book_id FROM wishlist
```

```

        GROUP BY user_id, book_id HAVING COUNT(*) > 1
    ) sub;
    IF v = 0 THEN
        RAISE NOTICE 'Testfall 2.1: PASS -- Keine doppelten (user_id, book_id) in
wishlist';
    ELSE
        RAISE NOTICE 'Testfall 2.1: FAIL -- % doppelte Kombinationen!', v;
    END IF;
END $$;

```

Output

AZ user ▼	123 wishlist_entry ▼
Anna Müller	2
Ben Schmidt	2
Ida Koch	1
Greta Hoffmann	1
Felix Becker	1
Lars Wolf	1
Clara Fischer	1
Eva Wagner	1
Katja Klein	1
David Weber	1
Hans Schäfer	1
Jonas Richter	1

2.6. Testfall 6 und Output

```

Updated Rows 0
Execute time 0.003s
Start time Wed Mar 11 15:01:39 CET 2026
Finish time Wed Mar 11 15:01:39 CET 2026
Query -----
-- Testfall 2.2: UNIQUE-Constraint -- Duplikat-Insert (Negativtest)
-----
DO $$ BEGIN RAISE NOTICE '--- Testfall 2.2: wishlist Duplikat-Insert (Negativtest) ---'; END $$;
DO $$
DECLARE v_uid INT; v_bid INT;
BEGIN
    SELECT user_id, book_id INTO v_uid, v_bid FROM wishlist LIMIT 1;
    IF v_uid IS NULL THEN
        RAISE NOTICE 'Testfall 2.2: ÜBERSPRUNGEN -- keine Daten in wishlist';
        RETURN;
    END IF;
    BEGIN
        INSERT INTO wishlist (user_id, book_id) VALUES (v_uid, v_bid);
        RAISE NOTICE 'Testfall 2.2: FAIL -- Duplikat wurde akzeptiert!';
    EXCEPTION WHEN unique_violation THEN
        RAISE NOTICE 'Testfall 2.2: PASS -- Duplikat korrekt abgelehnt (unique_violation)';
    END;
END $$

```

2.7. Testfall 7

```
-- -----
-- Testfall 3.1: TERNÄR loans
-- users (borrower) x book_copies x time_slots
-- -----
DO $$ BEGIN RAISE NOTICE '--- Testfall 3.1: Ternäre Beziehung LOANS ---'; END $$;

SELECT
    l.loan_id,
    u.first_name || ' ' || u.last_name AS loaner,
    b.title AS book,
    CASE ts.day_of_week
        WHEN 0 THEN 'Mo' WHEN 1 THEN 'Di' WHEN 2 THEN 'Mi'
        WHEN 3 THEN 'Do' WHEN 4 THEN 'Fr' WHEN 5 THEN 'Sa'
        ELSE 'So'
    END AS weekday,
    ts.start_time::TEXT || ' - ' || ts.end_time::TEXT AS timeframe,
    l.status
FROM loans l
JOIN users u ON u.user_id = l.borrower_id
JOIN book_copies bc ON bc.copy_id = l.copy_id
JOIN books b ON b.book_id = bc.book_id
JOIN time_slots ts ON ts.slot_id = l.slot_id
ORDER BY l.loan_id;

DO $$
DECLARE v INT;
BEGIN
    SELECT COUNT(*) INTO v
    FROM loans l
    JOIN users u ON u.user_id = l.borrower_id
    JOIN book_copies bc ON bc.copy_id = l.copy_id
    JOIN books b ON b.book_id = bc.book_id
    JOIN time_slots ts ON ts.slot_id = l.slot_id;
    IF v > 0 THEN
        RAISE NOTICE 'Testfall 3.1: PASS -- % Ausleihen vollständig über 4 Tabellen
verknüpft', v;
    ELSE
        RAISE NOTICE 'Testfall 3.1: HINWEIS -- Keine Ausleihen in Testdaten';
    END IF;
END $$;
```

Ouput

123 loan_id	A-Z loaner	A-Z book	A-Z weekday	A-Z timeframe	A-Z status
1	Ben Schmidt	Steppenwolf	Di	09:00:00 - 12:00:00	returned
2	Clara Fischer	Perfume: The Story of a Murderer	Mi	14:00:00 - 17:00:00	returned
3	David Weber	The Name of the Rose	Do	10:00:00 - 13:00:00	returned
4	Felix Becker	Bahnwärter Thiel	Sa	09:00:00 - 11:00:00	returned
5	Greta Hoffmann	Homo Faber	So	11:00:00 - 14:00:00	returned
6	Hans Schäfer	Sophie's World	Mo	10:00:00 - 15:00:00	overdue
7	Jonas Richter	The Metamorphosis	Mi	08:00:00 - 10:00:00	returned
8	Anna Müller	Buddenbrooks	Do	13:00:00 - 16:00:00	active
9	Ben Schmidt	The Alchemist	Fr	17:00:00 - 20:00:00	active
10	Clara Fischer	To Kill a Mockingbird	Sa	09:00:00 - 12:00:00	active
11	David Weber	Steppenwolf	Di	09:00:00 - 12:00:00	cancelled
12	Eva Wagner	The Name of the Rose	Do	10:00:00 - 13:00:00	active

2.8. Testfall 8

```
-----
-- Testfall 3.2: TERNÄR ratings
-- users (reviewer) x books x loans
-----
DO $$ BEGIN RAISE NOTICE '--- Testfall 3.2: Ternäre Beziehung RATINGS ---'; END $$;

SELECT
    r.rating_id,
    u.first_name || ' ' || u.last_name AS reviewer,
    b.title AS book,
    r.score AS rating,
    l.loan_id AS loan_id,
    r.rated_at::DATE AS date
FROM ratings r
JOIN users u ON u.user_id = r.reviewer_id
JOIN books b ON b.book_id = r.book_id
JOIN loans l ON l.loan_id = r.loan_id
ORDER BY r.rating_id;

DO $$
DECLARE v INT;
BEGIN
    SELECT COUNT(*) INTO v
    FROM ratings r
    JOIN users u ON u.user_id = r.reviewer_id
    JOIN books b ON b.book_id = r.book_id
    JOIN loans l ON l.loan_id = r.loan_id;
    IF v > 0 THEN
        RAISE NOTICE 'Testfall 3.2: PASS -- % Bewertungen vollständig über 4 Tabellen
verknüpft', v;
    ELSE
        RAISE NOTICE 'Testfall 3.2: HINWEIS -- Keine Bewertungen in Testdaten';
    END IF;
END $$;
```

Ouput

123 rating_id	A-Z reviewer	A-Z book	123 rating	123 loan_id	🕒 date
1	Ben Schmidt	Steppenwolf	5	1	2026-03-11
2	Clara Fischer	Perfume: The Story of a Murderer	4	2	2026-03-11
3	David Weber	The Name of the Rose	5	3	2026-03-11
4	Felix Becker	Bahnwärter Thiel	3	4	2026-03-11
5	Greta Hoffmann	Homo Faber	4	5	2026-03-11
6	Jonas Richter	The Metamorphosis	4	7	2026-03-11
7	Anna Müller	The Alchemist	5	9	2026-03-11
8	Ben Schmidt	Buddenbrooks	4	10	2026-03-11
9	Clara Fischer	To Kill a Mockingbird	5	11	2026-03-11
10	Eva Wagner	The Name of the Rose	4	12	2026-03-11

2.9. Testfall 9

```
-----
-- Testfall 3.3: TERNÄR messages -- Self-Join auf users
-- users (sender) x users (receiver) x book_copies
-----
DO $$ BEGIN RAISE NOTICE '--- Testfall 3.3: Ternäre Beziehung MESSAGES (Self-Join) ---';
END $$;

SELECT
    m.message_id,
    s.first_name || ' ' || s.last_name AS sender,
    e.first_name || ' ' || e.last_name AS receiver,
    b.title AS book,
    m.subject,
    m.is_read
FROM messages m
JOIN users s ON s.user_id = m.sender_id
JOIN users e ON e.user_id = m.receiver_id
JOIN book_copies bc ON bc.copy_id = m.copy_id
JOIN books b ON b.book_id = bc.book_id
ORDER BY m.sent_at DESC;

DO $$
DECLARE v INT;
BEGIN
    SELECT COUNT(*) INTO v
    FROM messages m
    JOIN users s ON s.user_id = m.sender_id
    JOIN users e ON e.user_id = m.receiver_id
    JOIN book_copies bc ON bc.copy_id = m.copy_id;
    IF v > 0 THEN
        RAISE NOTICE 'Testfall 3.3: PASS -- % Nachrichten vollständig verknüpft (Self-Join bestätigt)', v;
    ELSE
        RAISE NOTICE 'Testfall 3.3: HINWEIS -- Keine Nachrichten in Testdaten';
    END IF;
END $$;
```

Output

123 message_id	AZ sender	AZ receiver	AZ book	AZ subject	☒ is_read
1	Ben Schmidt	Anna Müller	Steppenwolf	Anfrage: Der Steppenwolf	[v]
2	Anna Müller	Ben Schmidt	Steppenwolf	Re: Anfrage: Der Steppenwolf	[v]
3	Clara Fischer	Ben Schmidt	Perfume: The Story of a Murderer	Buchzustand Das Parfum	[]
4	David Weber	Clara Fischer	The Name of the Rose	Abholung Der Name der Rose	[v]
5	Eva Wagner	David Weber	The Hitchhiker's Guide to the Galaxy	Versand möglich?	[]
6	Felix Becker	Eva Wagner	Bahnwärter Thiel	Verlängerung möglich?	[v]
7	Greta Hoffmann	Felix Becker	Homo Faber	Danke für das Buch	[v]
8	Hans Schäfer	Greta Hoffmann	Sophie's World	Sofies Welt – Verfügbarkeit	[]
9	Ida Koch	Hans Schäfer	Dune	Dune – Rückgabe	[v]
10	Jonas Richter	Ida Koch	The Metamorphosis	Buchempfehlung	[]
11	Katja Klein	Jonas Richter	Buddenbrooks	Leihfrist Buddenbrooks	[v]
12	Lars Wolf	Anna Müller	The Alchemist	Abholung in Hannover	[]

2.10. Testfall 10 und Output

```
Updated Rows 0
Execute time 0.01s
Start time   Wed Mar 11 15:15:41 CET 2026
Finish time  Wed Mar 11 15:15:41 CET 2026
Query
-----
-- Testfall 4.1: book_copies mit ungültigem owner_id
-----
DO $$ BEGIN RAISE NOTICE '--- Testfall 4.1: FK book_copies.owner_id (Negativtest) ---'; END $$;
DO $$
BEGIN
    INSERT INTO book_copies (book_id, owner_id, condition_id)
    VALUES (1, 99999, 1);
    RAISE NOTICE 'Testfall 4.1: FAIL -- Ungültige owner_id wurde akzeptiert!';
EXCEPTION WHEN foreign_key_violation THEN
    RAISE NOTICE 'Testfall 4.1: PASS -- FK-Verletzung korrekt abgelehnt (owner_id 99999 existiert nicht)';
END $$
```

2.11. Testfall 11 und Ouput

```
Updated Rows 0
Execute time 0.007s
Start time   Wed Mar 11 15:18:04 CET 2026
Finish time  Wed Mar 11 15:18:04 CET 2026
Query
-----
-- Testfall 4.2: loans mit ungültigem borrower_id
-----
DO $$ BEGIN RAISE NOTICE '--- Testfall 4.2: FK loans.borrower_id (Negativtest) ---'; END $$;
DO $$
BEGIN
    INSERT INTO loans (copy_id, borrower_id, slot_id, due_date)
    VALUES (1, 88888, 1, CURRENT_DATE + 14);
    RAISE NOTICE 'Testfall 4.2: FAIL -- Ungültige borrower_id wurde akzeptiert!';
EXCEPTION WHEN foreign_key_violation THEN
    RAISE NOTICE 'Testfall 4.2: PASS -- FK-Verletzung korrekt abgelehnt (borrower_id 88888 existiert nicht)';
END $$
```

2.12. Testfall 12 und Output

```
Updated Rows 0
Execute time 0.006s
Start time   Wed Mar 11 15:19:07 CET 2026
Finish time  Wed Mar 11 15:19:07 CET 2026
Query
-----
-- Testfall 4.3: ratings mit ungültigem loan_id
-----
DO $$ BEGIN RAISE NOTICE '--- Testfall 4.3: FK ratings.loan_id (Negativtest) ---'; END $$;
DO $$
BEGIN
    INSERT INTO ratings (loan_id, reviewer_id, book_id, score)
    VALUES (77777, 1, 1, 4);
    RAISE NOTICE 'Testfall 4.3: FAIL -- Ungültige loan_id wurde akzeptiert!';
EXCEPTION WHEN foreign_key_violation THEN
    RAISE NOTICE 'Testfall 4.3: PASS -- FK-Verletzung korrekt abgelehnt (loan_id 77777 existiert nicht)';
END $$
```

2.13. Testfall 13 und Output

```
Updated Rows 0
Execute time 0.006s
Start time Wed Mar 11 15:20:19 CET 2026
Finish time Wed Mar 11 15:20:19 CET 2026
Query -----
-- Testfall 5.1: score außerhalb 1-5 in ratings
-----

DO $$ BEGIN RAISE NOTICE '--- Testfall 5.1: CHECK score BETWEEN 1 AND 5 ---'; END $$;
DO $$
DECLARE v_loan_id INT; v_uid INT; v_bid INT;
BEGIN
    SELECT l.loan_id, l.borrower_id, bc.book_id
    INTO v_loan_id, v_uid, v_bid
    FROM loans l
    JOIN book_copies bc ON bc.copy_id = l.copy_id
    WHERE l.loan_id NOT IN (SELECT loan_id FROM ratings)
    LIMIT 1;
    IF v_loan_id IS NULL THEN
        RAISE NOTICE 'Testfall 5.1: ÜBERSPRUNGEN -- keine unbewertete Ausleihe verfügbar';
        RETURN;
    END IF;
    BEGIN
        INSERT INTO ratings (loan_id, reviewer_id, book_id, score)
        VALUES (v_loan_id, v_uid, v_bid, 6);
        RAISE NOTICE 'Testfall 5.1: FAIL -- score=6 wurde akzeptiert!';
    EXCEPTION WHEN check_violation THEN
        RAISE NOTICE 'Testfall 5.1: PASS -- CHECK-Constraint korrekt ausgelöst (score=6 abgelehnt)';
    END;
END $$
```

2.14. Testfall 14 und Output

```
Updated Rows 0
Execute time 0.004s
Start time Wed Mar 11 15:22:23 CET 2026
Finish time Wed Mar 11 15:22:23 CET 2026
Query -----
-- Testfall 5.2: day_of_week ausserhalb 0-6
-----

DO $$ BEGIN RAISE NOTICE '--- Testfall 5.2: CHECK day_of_week BETWEEN 0 AND 6 ---'; END $$;
DO $$
BEGIN
    INSERT INTO time_slots (copy_id, day_of_week, start_time, end_time)
    VALUES (1, 8, '10:00', '12:00');
    RAISE NOTICE 'Testfall 5.2: FAIL -- day_of_week=8 wurde akzeptiert!';
EXCEPTION WHEN check_violation THEN
    RAISE NOTICE 'Testfall 5.2: PASS -- CHECK-Constraint korrekt ausgelöst (day_of_week=8 abgelehnt)';
END $$
```

2.15. Testfall 15 und Output

```
Updated Rows 0
Execute time 0.005s
Start time   Wed Mar 11 15:24:07 CET 2026
Finish time  Wed Mar 11 15:24:07 CET 2026
Query
-----
-- Testfall 5.3: end_time <= start_time
-----
DO $$ BEGIN RAISE NOTICE '--- Testfall 5.3: CHECK end_time > start_time ---'; END $$;
DO $$
BEGIN
    INSERT INTO time_slots (copy_id, day_of_week, start_time, end_time)
    VALUES (1, 1, '14:00', '10:00');
    RAISE NOTICE 'Testfall 5.3: FAIL -- end_time < start_time wurde akzeptiert!';
EXCEPTION WHEN check_violation THEN
    RAISE NOTICE 'Testfall 5.3: PASS -- CHECK-Constraint korrekt ausgelöst (end <= start abgelehnt)';
END $$
```

2.16. Testfall 16 und Output

```
Updated Rows 0
Execute time 0.005s
Start time   Wed Mar 11 15:24:52 CET 2026
Finish time  Wed Mar 11 15:24:52 CET 2026
Query
-----
-- Testfall 5.4: ungültigem status in loans
-----
DO $$ BEGIN RAISE NOTICE '--- Testfall 5.4: CHECK status IN (...) ---'; END $$;
DO $$
BEGIN
    UPDATE loans SET status = 'ungültig' WHERE loan_id = 1;
    RAISE NOTICE 'Testfall 5.4: FAIL -- status=ungültig wurde akzeptiert!';
EXCEPTION WHEN check_violation THEN
    RAISE NOTICE 'Testfall 5.4: PASS -- CHECK-Constraint korrekt ausgelöst (status=ungültig abgelehnt)';
END $$
```

2.17. Testfall 17

```
-----  
-- Testfall 6.1: Jedes Exemplar hat gültigen Besitzer  
-----  
DO $$ BEGIN RAISE NOTICE '--- Testfall 6.1: Jedes book_copy hat gültigen owner ---'; END  
$$;  
  
DO $$  
DECLARE v INT;  
BEGIN  
    SELECT COUNT(*) INTO v  
    FROM book_copies bc  
    LEFT JOIN users u ON u.user_id = bc.owner_id  
    WHERE u.user_id IS NULL;  
    IF v = 0 THEN  
        RAISE NOTICE 'Testfall 6.1: PASS -- Alle Exemplare haben einen gültigen  
Besitzer';  
    ELSE  
        RAISE NOTICE 'Testfall 6.1: FAIL -- % verwaiste Exemplare!', v;  
    END IF;  
END $$;
```

Ouput

```
--- Testfall 6.1: Jedes book_copy hat gültigen owner ---  
Testfall 6.1: PASS -- Alle Exemplare haben einen gültigen  
Besitzer
```

2.18. Testfall 18

```
-----  
-- Testfall 6.2: sender_id != receiver_id in messages  
-----  
DO $$ BEGIN RAISE NOTICE '--- Testfall 6.2: sender != receiver in messages ---'; END $$;  
  
DO $$  
DECLARE v INT;  
BEGIN  
    SELECT COUNT(*) INTO v FROM messages WHERE sender_id = receiver_id;  
    IF v = 0 THEN  
        RAISE NOTICE 'Testfall 6.2: PASS -- Kein Nutzer hat sich selbst eine Nachricht  
gesendet';  
    ELSE  
        RAISE NOTICE 'Testfall 6.2: FAIL -- % Nachrichten mit sender = receiver!', v;  
    END IF;  
END $$;  
  
DO $$ BEGIN RAISE NOTICE '--- Testfall 6.2b: Selbst-Nachricht einfügen (Negativtest) ---  
'; END $$;  
  
DO $$  
DECLARE v_uid INT; v_cid INT;  
BEGIN  
    SELECT sender_id, copy_id INTO v_uid, v_cid FROM messages LIMIT 1;  
    IF v_uid IS NULL THEN  
        RAISE NOTICE 'Testfall 6.2b: ÜBERSPRUNGEN -- keine messages vorhanden';  
        RETURN;  
    END IF;  
    BEGIN  
        INSERT INTO messages (sender_id, receiver_id, copy_id, subject, body)  
        VALUES (v_uid, v_uid, v_cid, 'Test', 'Selbst-Nachricht');  
        RAISE NOTICE 'Testfall 6.2b: FAIL -- Selbst-Nachricht wurde akzeptiert!';  
    EXCEPTION WHEN check_violation THEN  
        RAISE NOTICE 'Testfall 6.2b: PASS -- CHECK-Constraint korrekt ausgelöst  
(sender=receiver abgelehnt)';  
    END;
```

```
END;  
END $$;
```

Output

```
--- Testfall 6.2: sender != receiver in messages ---  
Testfall 6.2: PASS -- Kein Nutzer hat sich selbst eine  
Nachricht gesendet  
--- Testfall 6.2b: Selbst-Nachricht einfügen (Negativtest) --  
-  
Testfall 6.2b: PASS -- CHECK-Constraint korrekt ausgelöst  
(sender=receiver abgelehnt)
```

2.19. Testfall 19

```
-----  
-- Testfall 6.3: due_date liegt nach loaned_at  
-----  
DO $$ BEGIN RAISE NOTICE '--- Testfall 6.3: due_date > loaned_at ---'; END $$;  
  
SELECT loan_id, loaned_at, due_date,  
       (due_date - loaned_at) AS loan_period  
FROM loans  
ORDER BY loan_id;  
  
DO $$  
DECLARE v INT;  
BEGIN  
    SELECT COUNT(*) INTO v FROM loans WHERE due_date <= loaned_at;  
    IF v = 0 THEN  
        RAISE NOTICE 'Testfall 6.3: PASS -- Alle Ausleihen haben due_date nach  
loaned_at';  
    ELSE  
        RAISE NOTICE 'Testfall 6.3: FAIL -- % Ausleihen mit ungültigem Datum!', v;  
    END IF;  
END $$;
```

Output

123 loan_id	🕒 loaned_at	🕒 due_date	123 loan_period
1	2024-01-10	2024-01-24	14
2	2024-02-05	2024-02-26	21
3	2024-03-01	2024-03-15	14
4	2024-04-10	2024-04-24	14
5	2024-05-15	2024-05-29	14
6	2024-06-01	2024-06-22	21
7	2024-07-20	2024-08-03	14
8	2024-08-05	2024-09-02	28
9	2024-08-20	2024-09-03	14
10	2024-09-01	2024-09-22	21
11	2024-09-10	2024-09-24	14
12	2024-10-01	2024-10-15	14

2.20. Testfall 20

```
-----  
-- Testfall 7.1: books.genre_id -> genres  
-----  
DO $$ BEGIN RAISE NOTICE '--- Testfall 7.1: books.genre_id -> genres ---'; END $$;  
  
DO $$  
DECLARE v INT;  
BEGIN  
    SELECT COUNT(*) INTO v  
    FROM books b  
    LEFT JOIN genres g ON g.genre_id = b.genre_id  
    WHERE b.genre_id IS NOT NULL AND g.genre_id IS NULL;  
    IF v = 0 THEN  
        RAISE NOTICE 'Testfall 7.1: PASS -- Alle Bücher haben gültiges Genre';  
    ELSE  
        RAISE NOTICE 'Testfall 7.1: FAIL -- % Bücher mit ungültigem genre_id!', v;  
    END IF;  
END $$;
```

Output

```
Testfall 7.1: PASS -- Alle Bücher haben gültiges Genre
```

2.21. Testfall 21

```
-----  
-- Testfall 7.2: books.language_id -> languages  
-----  
DO $$ BEGIN RAISE NOTICE '--- Testfall 7.2: books.language_id -> languages ---'; END $$;  
  
DO $$  
DECLARE v INT;  
BEGIN  
    SELECT COUNT(*) INTO v  
    FROM books b  
    LEFT JOIN languages l ON l.language_id = b.language_id  
    WHERE b.language_id IS NOT NULL AND l.language_id IS NULL;  
    IF v = 0 THEN  
        RAISE NOTICE 'Testfall 7.2: PASS -- Alle Bücher haben gültige Sprache';  
    ELSE  
        RAISE NOTICE 'Testfall 7.2: FAIL -- % Bücher mit ungültiger language_id!', v;  
    END IF;  
END $$;
```

Output

```
--- Testfall 7.2: books.language_id -> languages ---  
Testfall 7.2: PASS -- Alle Bücher haben gültige Sprache
```

2.22. Testfall 22

```
-----  
-- Testfall 7.3: book_copies.condition_id -> book_conditions  
-----  
DO $$ BEGIN RAISE NOTICE '--- Testfall 7.3: book_copies.condition_id -> book_conditions  
---'; END $$;  
  
DO $$  
DECLARE v INT;  
BEGIN  
    SELECT COUNT(*) INTO v  
    FROM book_copies bc  
    LEFT JOIN book_conditions c ON c.condition_id = bc.condition_id  
    WHERE c.condition_id IS NULL;  
    IF v = 0 THEN  
        RAISE NOTICE 'Testfall 7.3: PASS -- Alle Exemplare haben gültigen Zustand';  
    ELSE  
        RAISE NOTICE 'Testfall 7.3: FAIL -- % Exemplare ohne gültigen Zustand!', v;  
    END IF;  
END $$;
```

Output

```
--- Testfall 7.3: book_copies.condition_id -> book_conditions  
---  
Testfall 7.3: PASS -- Alle Exemplare haben gültigen Zustand
```


2.23. Genereller Output

```
Testfall 1.1: Nutzer mit Exemplaren: 12, davon mit >1
Exemplar: 1
Testfall 1.1: PASS -- 1 Nutzer besitzen mehr als 1 Exemplar
(1:N bestätigt)
--- Testfall 1.1: users -> book_copies (1:N) ---
Testfall 1.1: Nutzer mit Exemplaren: 12, davon mit >1
Exemplar: 1
Testfall 1.1: PASS -- 1 Nutzer besitzen mehr als 1 Exemplar
(1:N bestätigt)
--- Testfall 1.3: book_copies -> time_slots (1:N) ---
Testfall 1.3: Exemplare mit Zeitslots: 12, davon mit >1 Slot:
0
Testfall 1.3: HINWEIS -- Jedes Exemplar hat genau 1 Zeitslot
in Testdaten (1:N möglich, aber nicht belegt)
--- Testfall 1.3: book_copies -> time_slots (1:N) ---
Testfall 1.3: Exemplare mit Zeitslots: 12, davon mit >1 Slot:
0
Testfall 1.3: HINWEIS -- Jedes Exemplar hat genau 1 Zeitslot
in Testdaten (1:N möglich, aber nicht belegt)
Testfall 1.4: PASS -- Keine loan_id hat mehr als 1 Bewertung
(1:1 bestätigt)
--- Testfall 1.4: loans -> ratings (1:1, UNIQUE) ---
Testfall 1.4: PASS -- Keine loan_id hat mehr als 1 Bewertung
(1:1 bestätigt)
--- Testfall 2.1: wishlist UNIQUE(user_id, book_id) ---
Testfall 2.1: PASS -- Keine doppelten (user_id, book_id) in
wishlist
--- Testfall 1.4: loans -> ratings (1:1, UNIQUE) ---
Testfall 1.4: PASS -- Keine loan_id hat mehr als 1 Bewertung
(1:1 bestätigt)
Testfall 2.1: PASS -- Keine doppelten (user_id, book_id) in
wishlist
--- Testfall 2.1: wishlist UNIQUE(user_id, book_id) ---
Testfall 2.1: PASS -- Keine doppelten (user_id, book_id) in
wishlist
--- Testfall 2.2: wishlist Duplikat-Insert (Negativtest) ---
Testfall 2.2: PASS -- Duplikat korrekt abgelehnt
(unique_violation)
--- Testfall 2.2: wishlist Duplikat-Insert (Negativtest) ---
Testfall 2.2: PASS -- Duplikat korrekt abgelehnt
(unique_violation)
Testfall 3.1: PASS -- 12 Ausleihen vollstaendig ueber 4
Tabellen verknuepft
--- Testfall 3.1: Ternäre Beziehung LOANS ---
Testfall 3.1: PASS -- 12 Ausleihen vollstaendig ueber 4
Tabellen verknuepft
--- Testfall 3.1: Ternäre Beziehung LOANS ---
```

```
Testfall 3.1: PASS -- 12 Ausleihen vollstaendig ueber 4
Tabellen verknuepft
--- Testfall 3.1: Ternäre Beziehung LOANS ---
Testfall 3.1: PASS -- 12 Ausleihen vollständig über 4
Tabellen verknüpft
--- Testfall 3.2: Ternäre Beziehung RATINGS ---
Testfall 3.2: PASS -- 10 Bewertungen vollständig über 4
Tabellen verknüpft
--- Testfall 3.3: Ternäre Beziehung MESSAGES (Self-Join) ---
Testfall 3.3: PASS -- 12 Nachrichten vollständig verknüpft
(Self-Join bestätigt)
--- Testfall 4.1: FK book_copies.owner_id (Negativtest) ---
Testfall 4.1: PASS -- FK-Verletzung korrekt abgelehnt
(owner_id 99999 existiert nicht)
--- Testfall 4.2: FK loans.borrower_id (Negativtest) ---
Testfall 4.2: PASS -- FK-Verletzung korrekt abgelehnt
(borrower_id 88888 existiert nicht)
--- Testfall 4.3: FK ratings.loan_id (Negativtest) ---
Testfall 4.3: PASS -- FK-Verletzung korrekt abgelehnt
(loan_id 77777 existiert nicht)
--- Testfall 5.1: CHECK score BETWEEN 1 AND 5 ---
Testfall 5.1: PASS -- CHECK-Constraint korrekt ausgelöst
(score=6 abgelehnt)
--- Testfall 5.2: CHECK day_of_week BETWEEN 0 AND 6 ---
Testfall 5.2: PASS -- CHECK-Constraint korrekt ausgelöst
(day_of_week=8 abgelehnt)
--- Testfall 5.3: CHECK end_time > start_time ---
Testfall 5.3: PASS -- CHECK-Constraint korrekt ausgelöst (end
<= start abgelehnt)
--- Testfall 5.4: CHECK status IN (...) ---
Testfall 5.4: PASS -- CHECK-Constraint korrekt ausgelöst
(status=ungültig abgelehnt)
--- Testfall 6.1: Jedes book_copy hat gültigen owner ---
Testfall 6.1: PASS -- Alle Exemplare haben einen gültigen
Besitzer
--- Testfall 6.2: sender != receiver in messages ---
Testfall 6.2: PASS -- Kein Nutzer hat sich selbst eine
Nachricht gesendet
--- Testfall 6.2b: Selbst-Nachricht einfügen (Negativtest) --
-
Testfall 6.2b: PASS -- CHECK-Constraint korrekt ausgelöst
(sender=receiver abgelehnt)
--- Testfall 6.3: due_date > loaned_at ---
Testfall 6.3: PASS -- Alle Ausleihen haben due_date nach
loaned_at
--- Testfall 6.3: due_date > loaned_at ---
Testfall 6.3: PASS -- Alle Ausleihen haben due_date nach
loaned_at
Testfall 7.1: PASS -- Alle Bücher haben gültiges Genre
```

```

--- Testfall 7.2: books.language_id -> languages ---
Testfall 7.2: PASS -- Alle Bücher haben gültige Sprache
--- Testfall 7.3: book_copies.condition_id -> book_conditions
---
Testfall 7.3: PASS -- Alle Exemplare haben gültigen Zustand
=====
TESTBLOCK 1: Kardinalitäten (1:N)
=====
--- Testfall 1.1: users -> book_copies (1:N) ---
Testfall 1.1: Nutzer mit Exemplaren: 12, davon mit >1
Exemplar: 1
Testfall 1.1: PASS -- 1 Nutzer besitzen mehr als 1 Exemplar
(1:N bestätigt)
--- Testfall 1.2: books -> book_copies (1:N) ---
Testfall 1.2: Gesamt Titel mit Exemplaren: 12, davon mit >1
Exemplar: 1
Testfall 1.2: PASS -- 1 Titel haben number_of_copies > 1 (1:N
bestätigt)
--- Testfall 1.3: book_copies -> time_slots (1:N) ---
Testfall 1.3: Exemplare mit Zeitslots: 12, davon mit >1 Slot:
0
Testfall 1.3: HINWEIS -- Jedes Exemplar hat genau 1 Zeitslot
in Testdaten (1:N möglich, aber nicht belegt)
--- Testfall 1.4: loans -> ratings (1:1, UNIQUE) ---
Testfall 1.4: PASS -- Keine loan_id hat mehr als 1 Bewertung
(1:1 bestätigt)
=====
TESTBLOCK 2: M:N Beziehung (wishlist)
=====
--- Testfall 2.1: wishlist UNIQUE(user_id, book_id) ---
Testfall 2.1: PASS -- Keine doppelten (user_id, book_id) in
wishlist
--- Testfall 2.2: wishlist Duplikat-Insert (Negativtest) ---
Testfall 2.2: PASS -- Duplikat korrekt abgelehnt
(unique_violation)
=====
TESTBLOCK 3: Ternäre Beziehungen
=====
--- Testfall 3.1: Ternäre Beziehung LOANS ---
Testfall 3.1: PASS -- 12 Ausleihen vollständig über 4
Tabellen verknüpft
--- Testfall 3.2: Ternäre Beziehung RATINGS ---
Testfall 3.2: PASS -- 10 Bewertungen vollständig über 4
Tabellen verknüpft
--- Testfall 3.3: Ternäre Beziehung MESSAGES (Self-Join) ---
Testfall 3.3: PASS -- 12 Nachrichten vollständig verknüpft
(Self-Join bestätigt)
=====
TESTBLOCK 4: Fremdschlüssel-Integrität

```

```

=====
--- Testfall 4.1: FK book_copies.owner_id (Negativtest) ---
Testfall 4.1: PASS -- FK-Verletzung korrekt abgelehnt
(owner_id 99999 existiert nicht)
--- Testfall 4.2: FK loans.borrower_id (Negativtest) ---
Testfall 4.2: PASS -- FK-Verletzung korrekt abgelehnt
(borrower_id 88888 existiert nicht)
--- Testfall 4.3: FK ratings.loan_id (Negativtest) ---
Testfall 4.3: PASS -- FK-Verletzung korrekt abgelehnt
(loan_id 77777 existiert nicht)
=====
TESTBLOCK 5: CHECK-Constraints
=====
--- Testfall 5.1: CHECK score BETWEEN 1 AND 5 ---
Testfall 5.1: PASS -- CHECK-Constraint korrekt ausgelöst
(score=6 abgelehnt)
--- Testfall 5.2: CHECK day_of_week BETWEEN 0 AND 6 ---
Testfall 5.2: PASS -- CHECK-Constraint korrekt ausgelöst
(day_of_week=8 abgelehnt)
--- Testfall 5.3: CHECK end_time > start_time ---
Testfall 5.3: PASS -- CHECK-Constraint korrekt ausgelöst (end
<= start abgelehnt)
--- Testfall 5.4: CHECK status IN (...) ---
Testfall 5.4: PASS -- CHECK-Constraint korrekt ausgelöst
(status=ungültig abgelehnt)
=====
TESTBLOCK 6: Datenkonsistenz
=====
--- Testfall 6.1: Jedes book_copy hat gültigen owner ---
Testfall 6.1: PASS -- Alle Exemplare haben einen gültigen
Besitzer
--- Testfall 6.2: sender != receiver in messages ---
Testfall 6.2: PASS -- Kein Nutzer hat sich selbst eine
Nachricht gesendet
--- Testfall 6.2b: Selbst-Nachricht einfügen (Negativtest) --
-
Testfall 6.2b: PASS -- CHECK-Constraint korrekt ausgelöst
(sender=receiver abgelehnt)
--- Testfall 6.3: due_date > loaned_at ---
Testfall 6.3: PASS -- Alle Ausleihen haben due_date nach
loaned_at
=====
TESTBLOCK 7: Normalisierung 3NF
=====
--- Testfall 7.1: books.genre_id -> genres ---
Testfall 7.1: PASS -- Alle Bücher haben gültiges Genre
--- Testfall 7.2: books.language_id -> languages ---
Testfall 7.2: PASS -- Alle Bücher haben gültige Sprache

```

```
--- Testfall 7.3: book_copies.condition_id -> book_conditions
---
Testfall 7.3: PASS -- Alle Exemplare haben gültigen Zustand
=====
TESTLAUF ABGESCHLOSSEN
Ergebnisse s. PASS / FAIL / HINWEIS oben

Block 1 -- Kardinalitäten 1:N    (4 Tests)
Block 2 -- M:N Beziehung          (2 Tests)
Block 3 -- Ternäre Beziehungen   (3 Tests)
Block 4 -- FK-Integrität          (3 Negativtests)
Block 5 -- CHECK-Constraints      (4 Negativtests)
Block 6 -- Datenkonsistenz        (3 Tests)
Block 7 -- Normalisierung 3NF     (3 Tests)
-----
Gesamt: 22 Testfälle
=====
```

3. Implementierungszusammenfassung

Die Datenbank der Buchtausch-App wurde in PostgreSQL implementiert und umfasst 12 Tabellen in der dritten Normalform (3NF). Alle Lookup-Werte (Genres, Sprachen, Verlage, Buchzustände) wurden in separate Tabellen ausgelagert, um Redundanz zu vermeiden. Die Kernentitäten users, books, book_copies und loans bilden das relationale Rückgrat des Systems. Drei ternäre Beziehungen wurden explizit modelliert: (1) Ausleihen (Ausleiher × Exemplar × Zeitslot), (2) Bewertungen (Rezensent × Buch × Ausleihe) und (3) Nachrichten (Sender × Empfänger × Exemplar). GPS-Koordinaten in users und book_copies ermöglichen eine bounding-box-basierte Näherungssuche, die ohne PostGIS-Extension auskommt. Alle Tabellen wurden mit mindestens 10 Dummy-Datensätzen befüllt. CHECK-Constraints sichern die Datenintegrität (z. B. Statuswerte in loans, Score-Bereich in ratings). Indizes auf Fremdschlüsseln verbessern die Abfrageperformance. Die SQL-Datei buchtausch.sql enthält alle CREATE TABLE-Statements, INSERT-Befehle und 8 Testabfragen inklusive Kommentare. Darüber hinaus sind alle descriptions, labels usw. standardmäßig auf Englisch hinterlegt und werden auf der Website je nach ausgewählter Sprache über i18n übersetzt.

3.1. Indizes

```
CREATE INDEX IF NOT EXISTS idx_book_copies_owner ON book_copies(owner_id);
CREATE INDEX IF NOT EXISTS idx_book_copies_book ON book_copies(book_id);
CREATE INDEX IF NOT EXISTS idx_loans_borrower ON loans(borrower_id);
CREATE INDEX IF NOT EXISTS idx_loans_copy ON loans(copy_id);
CREATE INDEX IF NOT EXISTS idx_loans_status ON loans(status);
CREATE INDEX IF NOT EXISTS idx_ratings_book ON ratings(book_id);
CREATE INDEX IF NOT EXISTS idx_messages_receiver ON messages(receiver_id);
CREATE INDEX IF NOT EXISTS idx_messages_sender ON messages(sender_id);
CREATE INDEX IF NOT EXISTS idx_wishlist_user ON wishlist(user_id);
CREATE INDEX IF NOT EXISTS idx_time_slots_copy ON time_slots(copy_id);
CREATE INDEX IF NOT EXISTS idx_book_authors_book ON book_authors(book_id);
CREATE INDEX IF NOT EXISTS idx_book_authors_author ON book_authors(author_id);
```

3.2. Trigger

```
CREATE OR REPLACE FUNCTION fn_update_overdue_loans()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE loans
    SET status = 'overdue'
    WHERE status = 'active'
        AND due_date < CURRENT_DATE
        AND returned_at IS NULL;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```

CREATE OR REPLACE TRIGGER trg_check_overdue
  AFTER INSERT ON loans
  FOR EACH STATEMENT
  EXECUTE FUNCTION fn_update_overdue_loans();

CREATE OR REPLACE FUNCTION fn_set_copy_unavailable()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.status = 'active' THEN
    UPDATE book_copies SET is_available = FALSE WHERE copy_id = NEW.copy_id;
  ELSIF NEW.status IN ('returned', 'cancelled') THEN
    UPDATE book_copies SET is_available = TRUE WHERE copy_id = NEW.copy_id;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER trg_loan_availability
  AFTER INSERT OR UPDATE OF status ON loans
  FOR EACH ROW
  EXECUTE FUNCTION fn_set_copy_unavailable();

```

3.3. Explain

```

-- Explain: Abfrage mit Index auf loans.borrower_id
EXPLAIN (FORMAT TEXT)
SELECT l.loan_id, u.first_name, u.last_name, b.title, l.status
FROM loans l
JOIN users u      ON u.user_id    = l.borrower_id
JOIN book_copies bc ON bc.copy_id  = l.copy_id
JOIN books b      ON b.book_id     = bc.book_id
WHERE l.borrower_id = 2;

```

Output

	AZ QUERY PLAN
1	Nested Loop (cost=12.97..34.83 rows=4 width=1014)
2	-> Nested Loop (cost=12.83..32.71 rows=4 width=502)
3	-> Index Scan using users_pkey on users u (cost=0.14..8.16 rows=1 width=440)
4	Index Cond: (user_id = 2)
5	-> Hash Join (cost=12.69..24.51 rows=4 width=70)
6	Hash Cond: (bc.copy_id = l.copy_id)
7	-> Seq Scan on book_copies bc (cost=0.00..11.30 rows=130 width=8)
8	-> Hash (cost=12.64..12.64 rows=4 width=70)
9	-> Bitmap Heap Scan on loans l (cost=4.18..12.64 rows=4 width=70)
10	Recheck Cond: (borrower_id = 2)
11	-> Bitmap Index Scan on idx_loans_borrower (cost=0.00..4.18 rows=4 width=0)
12	Index Cond: (borrower_id = 2)
13	-> Index Scan using books_pkey on books b (cost=0.14..0.53 rows=1 width=520)
14	Index Cond: (book_id = bc.book_id)

```
-- Explain: Bounding-Box-Näherungssuche (ohne PostGIS)
EXPLAIN (FORMAT TEXT)
SELECT bc.copy_id, b.title, u.first_name,
       bc.pickup_latitude, bc.pickup_longitude
FROM book_copies bc
JOIN books b ON b.book_id = bc.book_id
JOIN users u ON u.user_id = bc.owner_id
WHERE bc.is_available = TRUE
      AND bc.pickup_latitude BETWEEN 51.0 AND 53.0
      AND bc.pickup_longitude BETWEEN 8.0 AND 14.0;
```

Output

	A2 QUERY PLAN
1	Nested Loop (cost=0.28..29.53 rows=1 width=766)
2	-> Nested Loop (cost=0.14..20.99 rows=1 width=552)
3	-> Seq Scan on book_copies bc (cost=0.00..12.60 rows=1 width=40)
4	Filter: (is_available AND (pickup_latitude >= 51.0) AND (pickup_latitude <= 53.0) AND (pickup_longitude >= 8.0) AND (pickup_longitude <= 14.0))
5	-> Index Scan using books_pkey on books b (cost=0.14..8.16 rows=1 width=520)
6	Index Cond: (book_id = bc.book_id)
7	-> Index Scan using users_pkey on users u (cost=0.14..8.16 rows=1 width=222)
8	Index Cond: (user_id = bc.owner_id)

3.4. Zusammenfassung

Value	Wert
Tabellen gesamt	14
Datensätze gesamt	~165
Ternäre Beziehungen	3
Fremdschlüsselbeziehung	18
Testabfragen	22
DBMS	PostgreSQL
Normalisierung	3NF

Tabellenstruktur im Überblick

Tabelle	Typ	Datensätze	Beschreibung
genres	Lookup	12	Buchgenres
languages	Lookup	12	Sprachen mit ISO-Code
publishers	Lookup	12	Verlage mit Land
book_conditions	Lookup	7	Buchzustände
authors	Entität	12	Autoren (NEU: 3NF-Verbesserung)
users	Entität	12	Nutzer inkl. GPS-Koordinaten
books	Entität	12	Bücher mit FK auf author_id
book_authors	M:N	12	Bücher <-> Autoren (NEU)
book_copies	Entität	13	Physische Exemplare
time_slots	Entität	14	Abholzeiträume (1:N zu Exemplaren)
loans	Ternär	12	Ausleihen: Nutzer x Exemplar x Slot
ratings	Ternär	10	Bewertungen: Nutzer x Buch x Ausleihe
messages	Ternär	12	Nachrichten (Self-Join auf users)
wishlist	M:N	14	Wunschliste: Nutzer <-> Bücher